

MUNICIPAL ROBOTICS · RESEARCH

NOVEMBER 2026 · DRAFT

Seeing Is Measuring

Editable CAD programs from photographs

CAM PEDERSEN · MUNICIPAL ROBOTICS

Cover: a printed enclosure with a rounded shell, a recessed bezel, a speaker grille and two round ports. Today's grammar cannot write it down (Section 7).

Abstract

A photograph of a part shows which features the part has and how large they are. We recover both as an editable CAD program, a plate and a list of holes, counterbores, slots, bosses and patterns, each with its dimensions, from three photographs beside a printed tag. Every photograph in this paper is synthetic, rendered with a phone camera model. We have not yet tested real photographs. We score a program by its description length in nats, which adds the residual, the parameter cost and the program length in one unit. The search keeps an edit that adds a feature only if the feature shortens the total, so a feature must pay for itself. Inside each edit, Levenberg–Marquardt fits the dimensions with a Jacobian from irys's edge-sampling estimator. The companion paper describes that renderer.

On 80 held-out synthetic parts drawn from the grammar, the recovery reads every dimension to within 1 mm on 37 (46%, Wilson 95% interval 36–57%). Most misses are wrong features: the recovery finds the right features on 44 parts, and 37 of those measure within 1 mm. A hole read as a counterbore, or the reverse, appears in 12 of the 36 parts with wrong features. The Fisher error bars are too small by a median factor of 6 and do not rank the errors. One empirical scale restores coverage on average but is not a per-dimension error bar.

A greedy search decides each feature with one yes-or-no comparison. We propose **feature amplitudes**, a continuous relaxation over which features a program has. Each optional feature gets an amplitude $a \in [0, 1]$. The image blends the renders without and with the feature, the derivative with respect to a is exact, and the description length charges aL. An anneal drives every amplitude to 0 or 1, and the fit then emits the ordinary program. We have tested it on three small problems only. It fits a hole from a start 2 mm off to within 0.03 mm. It removes a hole the target does not have, but only after we extended the anneal. On one part where the greedy search gave different answers in two runs, it chooses the hole over the counterbore. We have not yet run it on the held-out parts, so we do not know whether it improves identification. The last section lists what the grammar needs before it can describe a real printed enclosure.

1 Introduction

To measure a part, you usually touch it with calipers, a coordinate-measuring machine or a gauge pin. A photograph touches nothing, yet it already contains the part's dimensions, mixed up with its pose, its finish, the light in the room and the lens. Inverse rendering separates them by finding the parameters whose rendering matches the photograph. For an engineered object those parameters are not free-form vertices or a neural field. They are the dimensions in a CAD model, the angles of a mechanism's joints and the finish of its surfaces. If a renderer can recover them, and say how well it knows each one, it is a measuring instrument.

An instrument has two obligations. It must be **exact**, so the same input gives the same reading. It must be **precise**, so its error is small and known. The companion paper[1] makes the renderer exact. Its replay is bit-identical to its forward pass, and its gradients match central differences down to the dimensions of a BRep. This paper measures precision, first on a model we already have (Section 3), then on a model we must find.

A drawing of a part is a short program. "A 74 by 67 by 17 mm plate with three 3 mm holes on a 17 mm circle" is a dozen numbers and a few words, and anyone can make the part from it. A photograph of the same part is millions of pixels. Turning the photograph back into the program takes two steps. **Identification** finds which features the part has. **Measurement** finds their dimensions. Measurement is continuous and has gradients. Identification is discrete and has none.

irys supplies the gradients for measurement. vcad, our BRep CAD kernel[2], builds each program's solid. This paper adds identification. We search over editable CAD programs, fit each candidate's dimensions through irys's estimator, and choose between candidates by **description length**[3,4]. The description length is the number of nats needed to encode the photographs given the program, plus the program and its parameters. The search adds a feature only when the feature shortens that total.

On parts the grammar can describe, measurement works and identification often fails. On 80 held-out synthetic parts, the recovery identifies the features correctly on 44 (55%), and 37 of those 44 measure within 1 mm. The largest single group of failures is a confusion between a hole and a counterbore. A greedy search decides each edit from one fit, and these two candidates render almost the same image. The photographs barely show a counterbore's floor, so a counterbore looks like a hole of its outer radius.

The second half of the paper gives that decision a gradient. **Feature amplitudes** relax the choice of features to a continuous one. Each optional feature gets an amplitude $a \in [0, 1]$. The model image is the render without the feature plus a times the difference the feature makes. The description length charges aL, and an anneal drives each amplitude to 0 or 1. The derivative with respect to an amplitude is exact and costs one extra render, with no edge sampling. We expect it to choose well among features that something else has proposed, the hole-or-counterbore choice in particular, since a counterbore of depth zero is a hole. We expect it to discover poorly. An amplitude placed away from every real feature gets no gradient toward one.

Our contributions are:

- **A warm-up with a known model.** Three joint angles and an anodised finish of a robot arm read from one render, to 0.07° and 0.005 (Section 3).
- **Recovery by description length through a differentiable renderer.** A search over a CAD grammar whose every candidate is fitted by Levenberg–Marquardt with an edge-sampled Jacobian and scored in nats, with three photographs and a printed tag as the only input (Section 4).
- **A held-out test.** A development set of 20 parts, then 80 new in-grammar parts and 24 out-of-grammar parts run once with every setting frozen, with per-feature error, failure modes and a check of the error bars against the actual errors (Section 6).
- **Feature amplitudes**, a continuous relaxation over the discrete part of a CAD program, with a continuous description length. We test it on three small problems and not yet on the held-out parts (Section 5, Section 6.4).

1.1 Scope and limits

- **The photographs are synthetic.** irys's OpenPBR scene kernel renders them with a phone camera model. No number in this paper comes from a real photograph (Section 7). The targets and the final path-traced fit share a renderer, so the results exclude every error a real camera, print and room would add. The search itself fits a cheaper photo model, which differs from the renderer that made the targets (Section 4).
- **The grammar is a plate.** One extruded block with a corner treatment and features cut into or standing on its top face. The out-of-grammar parts of Table 2 test its edges, not a different family.
- **The search is the method's weakest part.** The amplitude relaxation improves how it chooses between proposals and does not change what it proposes.
- **The arm fit is one run.** It converges from a start 23° off at the elbow. Under sky-only light the image loss along the elbow has a basin only about ±5° wide, and a fit without the sun stopped in a local minimum. We have not measured convergence over starts or seeds.
- **The search is not deterministic across machines.** One part gave a different program on a second run (Section 6.4). We have not measured this variance.
- **Compute.** A part takes about ten minutes of CPU time (Section 6). Only the final path-traced fit uses the companion paper's GPU path.

2 Related work

Inverse CAD and program synthesis. Recovering an editable CAD description is usually posed as program synthesis. CSGNet parses shapes into CSG programs with a recurrent network[5]; InverseCSG converts meshes to compact CSG trees with a solver-based synthesizer[6]; Szalinski rewrites flat CSG into programs with loops by equality saturation[7]; DeepCAD generates sketch-and-extrude sequences[8]; ShapeAssembly writes part-assembly programs[9]; Point2CAD recovers BRep surfaces from point clouds[10], and CAD-Recode writes CAD code from point clouds with a language

model[11]. Ellis et al. infer graphics programs from images by proposing primitives and synthesizing a program over them[12], and DreamCoder learns a library of program components alongside the search[13]. Learned systems train on construction sequences such as the Fusion 360 Gallery[14], and Point2Cyl segments point clouds into extrusion cylinders[15]. Tian et al. infer and execute 3D shape programs[16], and Ellis et al. guide program search by executing partial programs[17]. These methods take meshes, point clouds or clean drawings, or learn a prior from a dataset. Ours starts from photographs and scores each candidate by rendering it. We do not compare against any of them in this draft (Section 8).

Analysis by synthesis. Fitting a generative graphics model to an image is an old idea. Picture expresses it as a probabilistic program over scenes[18], and Cascaval et al. differentiate CAD programs so that edits to the shape move the parameters[19]. Leclerc used description length to choose among image descriptions[20], and the Bayesian information criterion[21] is the usual simpler penalty that our Laplace term refines.

Measurement from images. Close-range photogrammetry measures manufactured objects from calibrated images, usually by triangulating targets or features and adjusting cameras and points together[22]. We share its tools (a calibration target, bundle-adjustment-style least squares, Fisher-information error estimates) but fit a parametric model of the whole part to its pixels rather than to sparse points, so a hole's diameter is read from its outline and shading rather than from markers.

Relaxing discrete structure. Continuous relaxations of a discrete choice are common where the choice has no gradient. DARTS mixes candidate operations with softmax weights and anneals to one[23]; the Gumbel-softmax samples from a relaxed categorical[24]; DiffCSG differentiates CSG by rasterization so the boolean structure can be optimized[25]. The amplitude's length term aL is an ℓ_1 penalty on a selection variable[26], and the anneal $\beta a(1-a)$ is the concave penalty that topology optimisation uses to push densities to 0 or 1[27] and that L_0 relaxations approximate[28]. The blend is a linear interpolation of two images, so its derivative is elementary. None of these parts is new. What we add is their use to choose CAD features. Two things make that use convenient. A feature's effect on the render is local and nearly additive, so the blend factors per feature and its gradient is exact. The description length sets the penalty's weight in the same unit as the residual, so there is no weight to tune.

Differentiable rendering. Edge sampling[29] and path replay[30], and their implementation in irys, are described in the companion paper[1]. Mitsuba 3[31] is the closest general system. Reparameterisation[32] and warped-area sampling[33] handle the same visibility discontinuities without sampling edges. The vcad BRep kernel differentiates CAD parameters upstream of the mesh[2].

3 A model you already have

We first recover the parameters of a model we already have. irys reads our humanoid's left arm, with its joints, from its CAD source. The unknowns are three joint angles and the finish of the anodised links, and the photograph is a path-traced render with fresh noise. Nothing in this problem is discrete, so it tests the continuous fit alone.



FIG 1 Recovering three joint angles and a finish from a render. The left arm of our humanoid robot, in context, 192×192, 8 samples per pixel, path depth 5 with next-event estimation. (a) The worst of the three joint errors for four configurations: all terms for 600 iterations, the arm alone in frame (the control), without secondary edges, and with a coarse-to-fine pyramid schedule. The last two diverge and leave the top of the plot within 50 iterations, so we keep secondary edges and drop the pyramid. (b) The all-terms run's joint angles against the truth (dashed). One run per configuration. `data/rose-fit.csv`, from irys `docs/phase2f-rose-loss.csv`.

The target is the arm at shoulder pitch, shoulder roll and elbow (−25.78, 16.04, −60.16)° with a bead-blasted clear-anodised finish $\theta = (0.30, 0.86, 0.88, 0.90)$. From (−8.6, 3.4, −37.2)° and a default finish, Adam on the image MSE for 600 iterations reaches (−25.73, 16.07, −60.23)°, an error of at most 0.07°, and $\theta = (0.303, 0.855, 0.876, 0.895)$, within 0.005 of the truth. The final MSE, 8.6×10^{-4}, equals that of the true parameters re-rendered with fresh noise (8.6×10^{-4}), so the fit has converged to the noise floor. An iteration costs 112 ms for the material gradient on the GPU, 167 ms for the primary edge term and 66 ms for the interior and secondary terms on the CPU; the run takes 244 s. The three terms sum to 345 ms per iteration, 207 s of the 244 s, and the rest is the forward render and bookkeeping. This is one run. We have not measured convergence over seeds or starting points.

This run started 23° from the answer at the elbow and converged. That is not general. With the sun removed, the image loss along the elbow has a basin about ±5° wide, and a fit stopped in a local minimum at −20°. A continuous fit needs a start inside its basin, and the discrete search in the next section has to supply one.

4 The recovery

4.1 Programs and their description length

A program is a plate (width, depth, height and a corner treatment: sharp, a fillet or a chamfer) and an ordered list of features on its top face: through-holes, counterbores, through-slots along either axis, bosses, linear hole patterns and circular hole patterns, each with its centre and dimensions, plus the part's pose on the table, (x, y, yaw). The program is built by the vcad kernel and tessellated for rendering. We score a program by its description length in nats,

$$\text{score}(\text{program}, \theta) = \underbrace{-\log p(\text{photos} \mid \text{render}(\text{program}, \theta))}_{\text{residual}} + \underbrace{\frac{1}{2} \log\det F(\theta)}_{\text{parameters}} + \underbrace{-\log p(\text{program})}_{\text{program}}$$

the residual under a robust (Huber) noise model, a Laplace parameter cost from the Fisher information $F = J^\top \Sigma^{-1} J$, and the program's own length under a uniform grammar prior, a few nats for each feature and its kind. Every term is in nats, so the score has no weight between fit and complexity. The method still has settings: the Huber threshold, the noise scaling below, a beam of two, four finalists, and for amplitudes a freeze threshold and an anneal schedule. We fixed all of them on the development set, except the anneal schedule, which we extended after a failure (Section 6.4). The parameter term uses the Fisher information, which Section 6 shows is miscalibrated by a factor of about 17.8, or $\ln 17.8 \approx 2.9$ nats per parameter. That is as large as a feature's program length, so the score's choices between programs with different parameter counts are biased by a comparable amount. We have not corrected for it. A feature that explains ten pixels of noise costs more in program and parameters than it saves in residual, so the search refuses it.

The noise model matters to the score more than to the fit. The residual of a good fit is not white. The appearance model cannot reproduce a window's reflection, interreflection or a tone curve, and those errors are smooth across many pixels. Counting those pixels as independent overcounts the nats, and the search then adds features to explain shading. We scale the noise variance by the residual's integrated autocorrelation area, which is the number of pixels per independent pixel, measured on the table away from the part.

4.2 Search

The search starts from a box. One silhouette cannot separate a plate's depth from its height, so the search fits one box per band of heights, fits each one's lighting and dimensions on the image, and keeps the box with the lowest robust cost. It then proposes edits: add a feature at a blob of the residual, remove a feature, turn a hole into a counterbore or a slot, merge holes into a pattern, or change the corner treatment. The search fits and scores each edit and keeps a beam of two. It refits the best four distinct programs at full resolution and makes the final choice there. It keeps an edit only if the edit shortens the description (Figure 2). When no edit does, the search stops and reports the dimensions with their error bars.

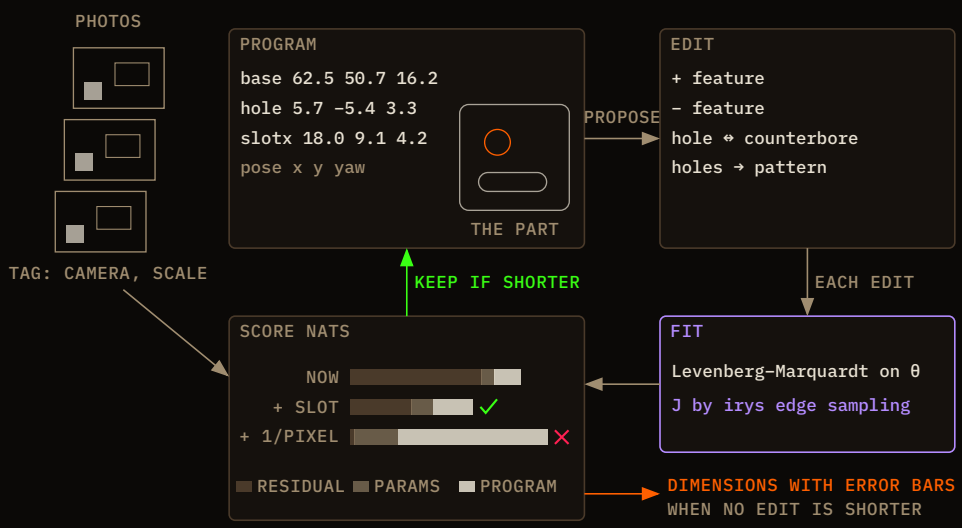


FIG 2 The recovery loop. The recovery edits an editable CAD program, fits each edit's dimensions with irys's edge-sampling Jacobian, scores it in nats, and keeps it only if the description of the photographs gets shorter. Bar lengths are schematic.

4.3 Fitting the dimensions

Inside each edit, Levenberg–Marquardt fits the continuous parameters (dimensions and pose) over a short blur homotopy, with a silhouette-distance term at the coarse levels and a chromatic coverage term at every level. The coverage term projects each pixel's colour onto the line between the backdrop's colour and the part's, which gives an observed coverage that shading does not change. The Jacobian is irys's primary edge-sampling estimator applied to every vertex's velocity under each parameter, the same estimator the companion paper validates against central differences, plus the moving-shadow (secondary) term for the key light.

The renderer inside the search is not the path tracer. A search makes thousands of fits, so it uses a deterministic **photo model**, which is a G-buffer with closed-form OpenPBR shading, spherical-harmonic environment light, one fitted key light with soft shadows, and ambient occlusion from 16 fixed rays per point. It is noise-free and about six times cheaper per evaluation. After the search, we refit the appearance under the path tracer and run the final fit on the path-traced residual, with path replay for the radiance and per-vertex edge sampling for the geometry, so in that configuration the reported program, score and dimensions are the path tracer's. We ran it only on the development set (Table 1, last column). The held-out results use the photo model throughout.

4.4 A derivative the search was missing

With three photographs the main remaining error on the development set was a confusion between holes and counterbores. We traced one case. The correct program reached the full-resolution finalists and lost by 27 nats to a counterbore whose depth never moved from its starting value. A counterbore's step shows in a photograph mostly as ambient occlusion on its floor. The photo model computed occlusion from a fixed set of 16 rays per point, which is a step function of the geometry with zero derivative almost everywhere. On a flat receiver those 16 directions are fixed in the world, so each is a directional light of weight 1/16, and its derivative is the moving-

shadow term the key light already had, with the receiver allowed to move too (a counterbore's floor moves with its depth). With that term the Jacobian's column for a counterbore's depth goes from cosine 0.88 against central differences to 0.97, every other column improves or holds, and the search no longer needs finite-difference columns for occluded parameters. The development set then reaches 15 of 20 parts within 1 mm with no finite differences at all, in 187 s per part against 409 s.

4.5 Camera and photographs

A printed AprilTag[34] (family tag36h11) beside the part fixes each camera's pose, its focal length (with the EXIF 35 mm-equivalent focal as a 2% prior) and the metric scale. We refine the tag's corners on linear light, because on sRGB values the edge shifts toward the dark side. The photographs share the part's pose, and each photograph gets its own appearance. The test photographs are synthetic. irys's OpenPBR scene kernel renders each part in PLA of one of six colours beside the tag sheet on grey paper, under a captured environment turned at random or a synthetic room with a window and a ceiling panel, plus a key light, at 2016×1512. A camera model then adds defocus, shot and read noise, a tone curve and JPEG compression, and writes an EXIF focal length.

5 Feature amplitudes

The search above is greedy over a discrete space. For every edit it fits the candidate, compares two scores and keeps one program. When two candidates explain the photographs almost equally well, the winner depends on where each fit started and stopped, and the search discards the loser. This section makes the choice continuous, so the fit that measures the features also chooses them.

5.1 The relaxation

Let a program be a base P_0 (the plate, its pose and any features already settled) and K optional features $f_1, \dots, f_K$ with dimensions θ_k. Give each optional feature an amplitude $a_k \in [0, 1]$ and render

$$I(a, \theta) = I_0(\theta_0) + \sum_{k=1}^{K} a_k \left(I_k(\theta_0, \theta_k) - I_0(\theta_0) \right)$$

where I_0 is the render of the base and I_k the render of the base with feature k added. At $a_k \in \{0, 1\}$ the blend is exactly the render of an ordinary program whenever the features' effects on the image do not overlap, which is the usual case for cuts on a plate. Two holes far apart change disjoint pixels, and the change one makes does not depend on the other. Between the endpoints the blend is not the render of any solid. It only has to connect two programs with a path along which we can differentiate the score. The silhouette (coverage) channels blend the same way.

5.2 Gradients

The derivative with respect to an amplitude is the difference the feature makes,

$$\frac{\partial I}{\partial a_k} = I_k - I_0,$$

which is exact and needs only the two renders the blend already uses, with no edge sampling, finite difference or step size. The dimensions' columns are the blend of the ordinary Jacobians,

$$\frac{\partial I}{\partial \theta_k} = a_k \frac{\partial I_k}{\partial \theta_k}, \quad \frac{\partial I}{\partial \theta_0} = \left(1 - \sum_k a_k\right) \frac{\partial I_0}{\partial \theta_0} + \sum_k a_k \frac{\partial I_k}{\partial \theta_0},$$

each from irys's edge-sampling estimator in its own branch. The image therefore constrains a feature's dimensions only in proportion to its amplitude, and at $a_k = 0$ their column is zero. We freeze a feature's dimensions when its amplitude is below 0.02.

5.3 A continuous description length

The score of Section 4 charges each feature its program length and its parameter cost. On the relaxation the charge becomes linear in the amplitude,

$$\mathcal{S}(a, \theta) = -\log p(\text{photos} \mid I(a, \theta)) + \sum_k a_k L_k + \beta \sum_k a_k (1 - a_k),$$

where L_k is the feature's description length (its kind and its parameters, in nats) and the last term is an anneal. At $\beta = 0$ the problem is a smooth fit of amplitudes and dimensions together, in which a feature's amplitude grows when the nats it saves in residual exceed the nats it costs. This is the discrete score's comparison in continuous form. As β grows, the concave penalty makes every interior amplitude cost more than either endpoint and drives the amplitudes to 0 or 1. The fit emits the base plus every feature whose amplitude rounds to 1. The search then refits and scores that program as an ordinary program, so the exact score still makes the final decision.

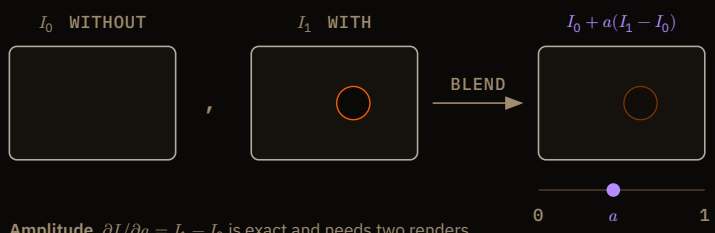
5.4 Alternatives on one face

The commonest failure in Section 6 is a choice between two alternatives at one place, a hole or a counterbore. Their effects overlap, so the blend does not factor between them. We keep their amplitudes on the simplex $a_{\text{hole}} + a_{\text{cbore}} \leq 1$, and the blend interpolates between the base, the hole and the counterbore. The counterbore's own dimensions give a second relaxation. A counterbore whose depth is zero, or whose outer radius equals its inner radius, is a hole, so the hole lies on the boundary of the counterbore's parameter domain. A fit that drives the depth to its bound has chosen the hole. The amplitude decides whether the step is worth its nats, and the depth decides where the step is.

5.5 What it should and should not do

The relaxation should be good at **choosing**. Given candidate features from the search or from a network, it fits them jointly with the base and with each other, and the residual's gradient decides which ones stay. It should be poor at **discovery**. An amplitude for a hole placed where the part has none makes the residual worse, so it goes to 0, which is correct. An amplitude for a hole placed a few radii from a real hole also goes to 0, because the two holes' edges do not overlap in the image and the edge term cannot move one onto the other. Amplitudes therefore need seeds, either the

residual blobs the greedy search already uses or a learned proposer. A seed that does not pay goes to 0, but every seed costs one more render per evaluation.



Amplitude. $\partial I/\partial a = I_1 - I_0$ is exact and needs two renders and no edge term.
Dimensions. $\partial I/\partial \theta = a\,\partial I_1/\partial \theta$, edge-sampled in the $a = 1$ branch. At $a = 0$ the column vanishes.
Length. The feature costs aL nats. An anneal $\beta\, a(1-a)$ drives a to 0 or 1, and the fit then emits the ordinary program.

A counterbore at depth 0 is a hole, so the hole lies on the boundary of the counterbore's parameters.

FIG 3 Feature amplitudes. The model image blends the render of the program without a feature and with it. The amplitude's derivative is the difference of the two renders, which is exact and needs no edge sampling. irys's edge-sampling estimator fits the feature's dimensions through the $a = 1$ branch. Schematic.

The implementation is a new module of the recovery crate (`crates/irys-recon/src/amp.rs`). It renders I_0 and each I_k with their Jacobians, assembles the blended residual and Jacobian with the robust weights of the ordinary fit, adds the length and anneal terms to the gradient (the anneal is concave and left out of the Gauss–Newton matrix), and runs damped Levenberg–Marquardt with the amplitudes clamped to $[0, 1]$ over a schedule $\beta \in \{0, 10, 100, 1000\}$ nats, extended to 10^4 and 10^5 in the runs reported last in Section 6.4.

6 Results

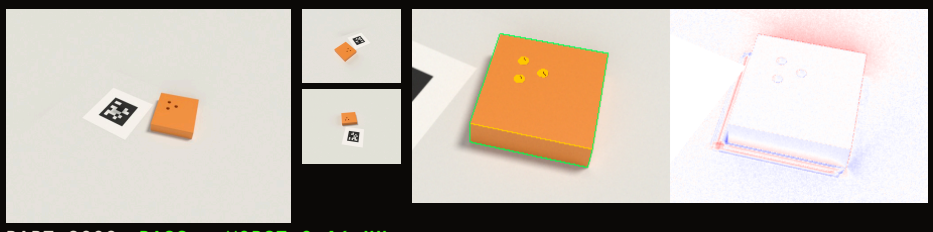
6.1 Development set

We developed the method on 20 parts drawn from the grammar, each recovered independently from one photograph or from three photographs taken from different directions (Table 1). Errors are the worst dimension of each part, after the plate's quarter-turn symmetry.

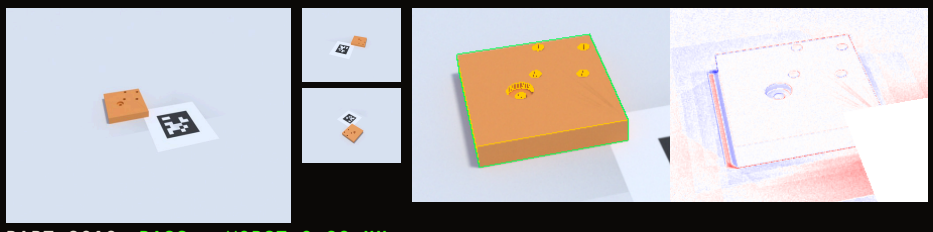
TABLE 1 The development set: dimensions from synthetic photographs, per part, worst dimension. With 20 parts the counts are coarse. The Wilson 95% interval on 15/20 is 53–89%, against 14–52% for one photograph. "+ AO grad" differentiates the photo model's ambient occlusion instead of estimating those columns by finite differences. In that column the parts that pass at 0.5 mm have worst-dimension errors from 0.16 to 0.38 mm (median 0.27). The last column runs the final fit through irys's path tracer. It gains one part at 0.5 mm and loses one at 1 mm, which at 20 parts is no measurable difference. Across all parts the pose on the tag plane is recovered to a median 0.13 mm and yaw within 0.07°, and the focal length within 0.6%.

RECOVERED	1 PHOTO	3 PHOTOS	+ AO GRAD	+ PATH-TRACED FIT
every dimension within 0.5 mm	2/20	10/20	10/20	11/20
every dimension within 1 mm	6/20	13/20	**15/20**	14/20
the right features (topology)	11/20	15/20	16/20	16/20

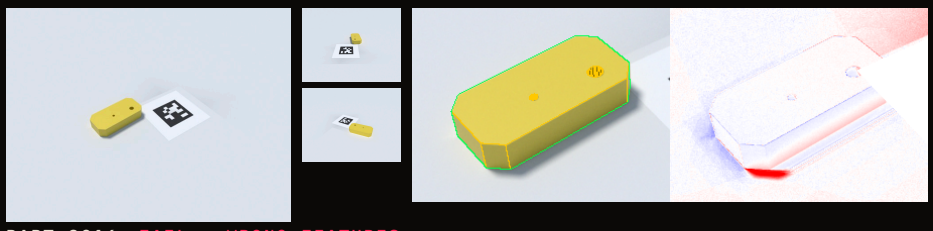
A single photograph cannot separate a plate's height from a boss's, and it does not show the floor of a counterbore. Three photographs usually resolve both. Figure 4 shows three of the parts with their photographs, the fit's overlay and the programs.



PART 3009 PASS · WORST 0.16 MM
TRUTH base 74.100 67.500 17.100; circular 3 10.400 10.300 8.600 32.200 3.000
FOUND base 73.998 67.343 16.980; circular 3 10.388 10.196 8.570 32.137 3.013



PART 3012 PASS · WORST 0.32 MM
TRUTH base 65.400 61.100 11.000; circular 4 15.900 -13.000 14.500 30.800 2.500; counterbore -1.500 7.900 2.400 5.600 3.100
FOUND base 65.348 60.992 10.774; circular 4 15.904 -12.950 14.496 120.775 2.494; counterbore -1.450 7.988 2.288 5.584 2.780



PART 3016 FAIL · WRONG FEATURES
TRUTH base 81.900 41.100 16.000 chamfer 7.800; hole -24.500 7.400 3.800; hole 7.600 3.200 2.000
FOUND base 81.490 40.885 15.903 chamfer 7.662; hole 7.560 3.272 2.027; counterbore -24.380 7.465 2.105 3.819 7.940

FIG 4 Three parts, three photographs each. Left, the synthetic photographs the recovery sees, each beside the printed tag. Right, the fit's overlay, which draws the recovered part's outline on the crop and the residual (red brighter, blue darker than the model). Part 3009 passes at 0.16 mm and 3012, with a four-hole circle and a counterbore, at 0.32 mm. A circle of four holes repeats every 90°, so 3012's recovered phase of 120.8° is the true 30.8°. Part 3016 fails. The recovery reads its first hole as a counterbore whose outer radius is the hole's, and Section 6.4 returns to this part. Programs are in millimetres, a plate `base w d h` and then each feature with its centre and radii. FOUND omits the pose. `data/recon-examples.json`.

6.2 Held-out parts

A development set overstates a method's accuracy, because the method was tuned on it. With every setting frozen, we ran the recovery once on 80 new grammar parts and on 24 parts outside the grammar. Table 2 reports the new parts alone, and also all 100 grammar parts including the 20 development parts. An out-of-grammar part's truth is still a program in our language, one the generator never draws, so "topology" means the same feature list as that program. The held-out runs use the photo model, with no final path-traced fit. The outside parts are six kinds of four: holes closer to

the edge than the generator allows, keyholes (two holes cut into each other), diagonal patterns, dense circles of small holes, every dimension off the 0.1 mm grid, and plates thicker than any in the grammar's range (Table 2).

TABLE 2 Held-out results. Three photographs, ambient occlusion differentiated, no finite differences, every setting frozen before the run. Wilson 95% intervals, in percent, in parentheses. The first row is the held-out result. The second adds the 20 development parts, which score 15 of 20 and so were a favourable draw. Among the new parts with the right topology the worst dimension has a median error of 0.39 mm (90th percentile 1.37). The last column conditions on the parts the search got right, so it describes the fit and not the method. Out of grammar, off-grid dimensions pass (3 of 4 within 1 mm) and keyholes never do (0 of 4). `data/measured.tsv`.

SET	≤ 0.5 MM	≤ 1 MM	TOPOLOGY	≤ 1 MM GIVEN TOPOLOGY
new in-grammar parts (80)	30/80 (28–48)	**37/80** (36–57)	44/80 (44–65)	37/44 (71–92)
with the 20 development parts (100)	40% (31–50)	52% (42–62)	60% (50–69)	87% (76–93)
out of grammar (24)	5/24	9/24	10/24	9/10

Within 1 mm falls from 15 of 20 on the development set to 37 of 80 on the new parts. Wrong features account for most of the fall: 36 of the 80 new parts have wrong features, and 7 more have the right features and a dimension off by more than 1 mm. 4 of the new parts fail before the search because the detector finds no tag in one of their photographs, on clean synthetic images, and they count as failures. The plate's height comes back 0.18 mm low on average, in 91% of parts, a bias we have not traced. A 1 mm threshold on plates of about 60 mm with 2 to 5 mm holes is loose for metrology, and we report no trivial baseline such as the grammar's mean part.

TABLE 3 How identification fails. The commonest modes among the 54 parts with wrong features, counted over all 124 parts (the new parts, the development parts and the out-of-grammar parts). A part can show several. A hole and counterbore confusion in either direction appears in 19 of the 54, and in 12 of the 36 wrong among the 80 new in-grammar parts. The last column is our interpretation and has not been tested.

FAILURE	PARTS	OUR READING (UNTESTED)
missed hole	11	a hole beside a larger feature, its residual merged into the other's
counterbore read as hole	10	the step's floor is shading the model gets least right
extra hole	10	a highlight or reflection explained as a cut
hole read as counterbore	9	a counterbore at the hole's radius, its inner hole too small
extra counterbore	7	as for extra holes, with the step buying shading
sharp corner read as fillet	7	a small fillet pays for a soft silhouette
extra boss	5	a highlight on the top face

When the features are right, accuracy varies by feature. The recovery reads hole radii to a median 0.038 mm and the radii in circular patterns to 0.019 mm, because a pattern's holes share one radius and pool their evidence. The worst dimension is a

slot's length (median 0.3 mm, 90th percentile 2.45, 11 of 29 above 0.5 mm). A slot's ends are short arcs seen obliquely, and nothing else constrains them along the slot's axis. Counterbore depths are next (median 0.23 mm, 90th percentile 0.78).

6.3 Error bars

The Fisher information gives each dimension a standard deviation. On these parts those are optimistic by a median factor of 6 and do not rank the errors (Spearman $\rho = -0.15$). The Fisher σ propagates the photographs' noise through the fit. The appearance model's bias dominates the error, and it moves an edge the same way in every photograph. One scale corrects the average coverage. Fitted on the even seeds of all 124 parts, $\sigma' = 17.8\sigma_F$ (bootstrap 12.7–21.8) covers 68% of the odd seeds' dimensions at $1\sigma'$, and the errors' heavy tails put the 95% interval at $\pm 6.72\sigma'$, which covers 96%. That split includes the development parts. On the 80 new in-grammar parts alone the same procedure gives a scale of 14, 65% coverage at $1\sigma'$ and a 95% interval of $\pm 9.15\sigma'$ (97% coverage), with $\rho = -0.12$. The scale is therefore known only to within about 14 to 18.

This is a weak result, and it limits the instrument claim. A scale fitted to give 68% coverage gives about 68% coverage on parts from the same distribution by construction. Because σ_F does not rank the errors, σ' does not tell a well-measured dimension from a poorly measured one. It is an average error with a unit, not an error bar. We suspect the cause is bias from the appearance model, since a fit to a target rendered by the photo model itself recovers the plate height to 0.04 mm, but we have not tested that on these parts.

6.4 Feature amplitudes: the smallest experiments

We test the relaxation on three small problems of increasing difficulty. Each is a single run, so none of them gives a rate.

(a) A feature that is there. A 60×40×8 mm plate with one 4 mm-radius through-hole, rendered by the path tracer in four known views (the sim-to-sim setting of the recovery's first version). The fit starts with the right plate and an optional hole at amplitude 0.5, 2 mm off in each coordinate and 1 mm small. It passes if the amplitude goes to 1 and every dimension ends within 0.5 mm. It passes. The amplitude rises in the first four steps while the a-weighted edge term moves the hole's position and radius to the target. The amplitude ends at 0.994 with every dimension within 0.024 mm, in 11 accepted steps and 108 renders (225 s on a shared machine). The amplitude stops 0.6% short of 1, because at $\beta \leq 1000$ nats the last 0.6% costs more in residual than the anneal's $\beta a(1-a)$ saves. Rounding emits the program.

(b) A feature that is not. The same start against a plate with no hole. It passes if the amplitude goes to 0. The emitted program is right (the plate alone, its dimensions within 0.01 mm), but the amplitude does not reach 0, so the run fails our criterion. It falls from 0.5 to 0.03 in the first step and stays there at $\beta \leq 10$. A hole at 3% amplitude is a faint grey disc, and the disc lowers the residual by about ten nats. The model's noise differs from the target's (the target is rendered at 64 samples per pixel, the model at 16), and the disc fits part of that difference. The anneal at $\beta = 1000$ lowers the amplitude to 0.014, and rounding emits no hole. The same failure can

occur at any scale. Between the endpoints the blend adds a free image component, and the fit uses it to explain whatever the model cannot. We then extended the anneal to $\beta = 10^4$ and 10^5 nats. This changes the schedule after seeing a failure, on the same two problems, so it is a fix and not a test. With it, (b)'s amplitude reaches 0 at $\beta = 10^4$, giving up about 30 nats of residual, and (a)'s reaches 1 with every dimension within 0.032 mm. Both runs then meet the criteria. The anneal must reach the size of the residual that a partial feature can absorb, which here is larger than the feature's description length.

(c) Hole or counterbore. Part 3016 of the development set, which the greedy search got wrong in the run of Figure 4: its 3.8 mm hole is recovered as a counterbore whose outer radius is the hole's and whose inner hole is too small. We take the search's own program, replace that counterbore by two alternatives at amplitudes 0.5 and 0.5 on the simplex, a through-hole of the counterbore's outer radius and the counterbore as found, and let the relaxation choose, using the photo model, robust noise model and appearance of the search's final fit. The relaxation chooses the hole. Its amplitude goes to 1 and the counterbore's to 0, with the hole's radius at 3.84 mm against the true 3.80 and its centre within 0.29 mm, in 93 s and 213 renders. The emitted program has the right topology. One caveat limits what this shows. On this run the greedy search found the hole itself (448 s), although the earlier run on another machine had read a counterbore. We therefore built the alternative from the earlier run's wrong reading, a counterbore of the hole's outer radius with that run's inner radius and depth. So the greedy answer differs between two runs on the same photographs, and we have not measured how often. The relaxation is meant to remove that instability, but this experiment does not show that it does. It shows one hand-built choice on one development part.

Not yet run. The experiment that would test the relaxation is the held-out set with the search's final choice made by amplitudes, against the greedy choice, on fresh parts and several seeds. We have not run it, and until we do the relaxation is a proposal. [TODO: amplitude finals against greedy on fresh held-out parts, several seeds each.]

7 Toward real objects (not yet done)

Every result above compares the recovery with a known program, because we rendered the photographs from the programs we try to recover. The next reference is a printed part measured with calipers. The kit for that experiment exists (a 50 mm tag sheet, five test parts exported for printing, and shooting and caliper instructions), and we have not run it. [TODO: print at least 20 parts, photograph each three times with a phone, recover, and compare with calipers.]

Real objects also need a larger grammar. The cover shows a 3D-printed enclosure for a small screen. To describe it, a program needs:

- **A shell.** A box whose edges each have their own radius (large on the top and front, small on the base), hollowed to a wall thickness. The plate grammar has one corner treatment for four vertical edges.

- **Features on any face.** The bezel, the grille and the ports are on the front, not the top. Every feature needs a host face and a frame on that face, and the search needs proposals for every visible face.
- **A recess feature.** The screen sits in a rectangular pocket with its own corner radius and depth, which is a counterbore generalized to a rounded rectangle.
- **Patterns as one feature.** The grille is about 150 small holes on a staggered grid. Written as 150 holes it costs 150 times the program length of one hole, more than it saves in residual. Written as one pattern (pitch, stagger, extent, hole radius) it costs a few nats, and its holes share one radius, as the circular patterns' holes already do (Section 6).
- **Masks for what the model cannot render.** The screen is a black mirror that reflects the room, and the shading model has no texture for the print's layer lines and seams. The recovery should down-weight such regions, as it already does for the tag, with a weight it estimates from the residual's local dispersion or the material's glossiness.

A larger grammar produces more proposals per round and more near-ties between them, such as a recess or a through-cut, or a grille of round holes or of hexagons. A greedy search fails on near-ties, and a relaxation that fits the alternatives together should not. Amplitudes do not help discovery, which is the larger problem on a real object. The search must first propose a feature on the right face before an amplitude can choose it.

8 Limitations

- **Synthetic photographs.** The only photographs in this paper are rendered, by the renderer the method is built on (Section 7).
- **No baselines.** We compare against no other method: not a BIC-scored search, a silhouette-only fit, multi-view stereo followed by primitive fitting, or a learned image-to-CAD model. We also do not ablate the noise scaling, the beam width or the program prior beyond the columns of Table 1.
- **One family of parts.** The out-of-grammar parts are small departures from it. Keyholes, which no single feature describes, fail every time.
- **Identification is weak.** The features are right on 44 of the 80 held-out parts, and we have tested the amplitude relaxation only on three small problems (Section 6.4).
- **The error bars are not yet error bars.** The Fisher σ does not rank the errors, and one empirical scale fixes only the average coverage (Section 6). The same Fisher term biases the score.
- **Bias.** The plate's height comes back 0.18 mm low on average for a reason we have not found.
- **Compute.** On the development set a part took a mean of 187 s on a 16-core laptop with the machine to itself. The 100 in-grammar parts took a median of 576 s of process time each, most of them on a 24-core machine running six processes of

four threads at once, which is about two minutes of wall time per part there. The relaxation adds one render per optional feature to every evaluation.

9 Reproducibility

The recovery is the `irys-recon` crate of irys, open source under Apache-2.0. Each row of `data/measured.tsv` names the irys report and commit that produced it. The 124-part run is irys commit `812dc62` (`docs/recon-v1-gate100-report.md` and its CSVs), regenerated by `scripts/recon_gate_stats.py`. The amplitude experiments are `examples/recon_amp.rs` on irys branch `claude/recon-amplitudes`, run once each on a 24-core Linux machine. They write their traces to `out/recon_amp/`, which we copied to `data/amp-*.json`.

10 Conclusion

The recovery measures well when it identifies the features, so identification is the next problem. On 80 held-out synthetic parts, a description-length search through a differentiable renderer reads every dimension of 37 parts to within a millimetre. It finds the right features on 44, and 37 of those measure within a millimetre. The largest group of wrong features is a hole confused with a counterbore (12 of 36 failures), and on one such part the greedy search gave different answers in two runs. Feature amplitudes give that choice a gradient. On three small problems the relaxation fits a feature it is given, removes a feature the target lacks once the anneal reaches 10^4 nats, and chooses the hole in one hand-built comparison. These are single runs. The paper still needs the amplitude run on fresh held-out parts, baselines, error bars that rank the errors, and photographs of real parts.

References

1. Pedersen, C. Differentiable Rendering with Exact Replay: irys, an Open-Source Spectral Path Tracer. (2026).
2. Pedersen, C. Exact Gradients Through a BRep Kernel. (2026).
3. Rissanen, J. Modeling by Shortest Data Description. *Automatica* **14,** 465–471 (1978).
4. Grünwald, P. D. *The Minimum Description Length Principle*. (MIT Press, 2007).
5. Sharma, G., Goyal, R., Liu, D., Kalogerakis, E. & Maji, S. CSGNet: Neural Shape Parser for Constructive Solid Geometry. in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5515–5523 (2018).
6. Du, T. *et al.* InverseCSG: Automatic Conversion of 3D Models to CSG Trees. *ACM Transactions on Graphics* **37,** (2018).
7. Nandi, C. *et al.* Synthesizing Structured CAD Models with Equality Saturation and Inverse Transformations. in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)* (2020).

8. Wu, R., Xiao, C. & Zheng, C. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. in *IEEE/CVF International Conference on Computer Vision (ICCV)* 6772–6782 (2021).
9. Jones, R. K. *et al.* ShapeAssembly: Learning to Generate Programs for 3D Shape Structure Synthesis. *ACM Transactions on Graphics* **39,** (2020).
10. Liu, Y., Obukhov, A., Wegner, J. D. & Schindler, K. Point2CAD: Reverse Engineering CAD Models from 3D Point Clouds. in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 3763–3772 (2024).
11. Rukhovich, D. *et al.* CAD-Recode: Reverse Engineering CAD Code from Point Clouds. (2024).
12. Ellis, K., Ritchie, D., Solar-Lezama, A. & Tenenbaum, J. B. Learning to Infer Graphics Programs from Hand-Drawn Images. in *Advances in Neural Information Processing Systems (NeurIPS)* vol. 31 (2018).
13. Ellis, K. *et al.* DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)* (2021). doi:10.1145/3453483.3454080.
14. Willis, K. D. D. *et al.* Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction from Human Design Sequences. *ACM Transactions on Graphics* **40,** (2021).
15. Uy, M. A. *et al.* Point2Cyl: Reverse Engineering 3D Objects from Point Clouds to Extrusion Cylinders. in *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2022).
16. Tian, Y. *et al.* Learning to Infer and Execute 3D Shape Programs. in *International Conference on Learning Representations* (2019).
17. Ellis, K. *et al.* Write, Execute, Assess: Program Synthesis with a REPL. in *Advances in Neural Information Processing Systems* (2019).
18. Kulkarni, T. D., Kohli, P., Tenenbaum, J. B. & Mansinghka, V. Picture: A Probabilistic Programming Language for Scene Perception. in *IEEE Conference on Computer Vision and Pattern Recognition* (2015).
19. Cascaval, D. *et al.* Differentiable 3D CAD Programs for Bidirectional Editing. *Computer Graphics Forum* **41,** (2022).
20. Leclerc, Y. G. Constructing Simple Stable Descriptions for Image Partitioning. *International Journal of Computer Vision* **3,** 73–102 (1989).
21. Schwarz, G. Estimating the Dimension of a Model. *The Annals of Statistics* **6,** 461–464 (1978).
22. Luhmann, T., Robson, S., Kyle, S. & Boehm, J. *Close-Range Photogrammetry and 3D Imaging*. (De Gruyter, 2019).
23. Liu, H., Simonyan, K. & Yang, Y. DARTS: Differentiable Architecture Search. in *International Conference on Learning Representations* (2019).

24. Jang, E., Gu, S. & Poole, B. Categorical Reparameterization with Gumbel-Softmax. in *International Conference on Learning Representations* (2017).

25. Yuan, H. *et al.* DiffCSG: Differentiable CSG via Rasterization. in *SIGGRAPH Asia 2024 Conference Papers* (2024).

26. Tibshirani, R. Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B* **58,** 267–288 (1996).

27. Bendsøe, M. P. & Sigmund, O. *Topology Optimization: Theory, Methods, And Applications*. (Springer, 2003).

28. Louizos, C., Welling, M. & Kingma, D. P. Learning Sparse Neural Networks through L_0 Regularization. in *International Conference on Learning Representations* (2018).

29. Li, T.-M., Aittala, M., Durand, F. & Lehtinen, J. Differentiable Monte Carlo Ray Tracing through Edge Sampling. *ACM Transactions on Graphics* **37,** (2018).

30. Vicini, D., Speierer, S. & Jakob, W. Path Replay Backpropagation: Differentiating Light Paths using Constant Memory and Linear Time. *ACM Transactions on Graphics* **40,** (2021).

31. Jakob, W. *et al.* Mitsuba 3 renderer. https://mitsuba-renderer.org/ (2022).

32. Loubet, G., Holzschuch, N. & Jakob, W. Reparameterizing Discontinuous Integrands for Differentiable Rendering. *ACM Transactions on Graphics* **38,** (2019).

33. Bangaru, S. P., Li, T.-M. & Durand, F. Unbiased Warped-Area Sampling for Differentiable Rendering. *ACM Transactions on Graphics* **39,** (2020).

34. Olson, E. AprilTag: A Robust and Flexible Visual Fiducial System. in *IEEE International Conference on Robotics and Automation* (2011).